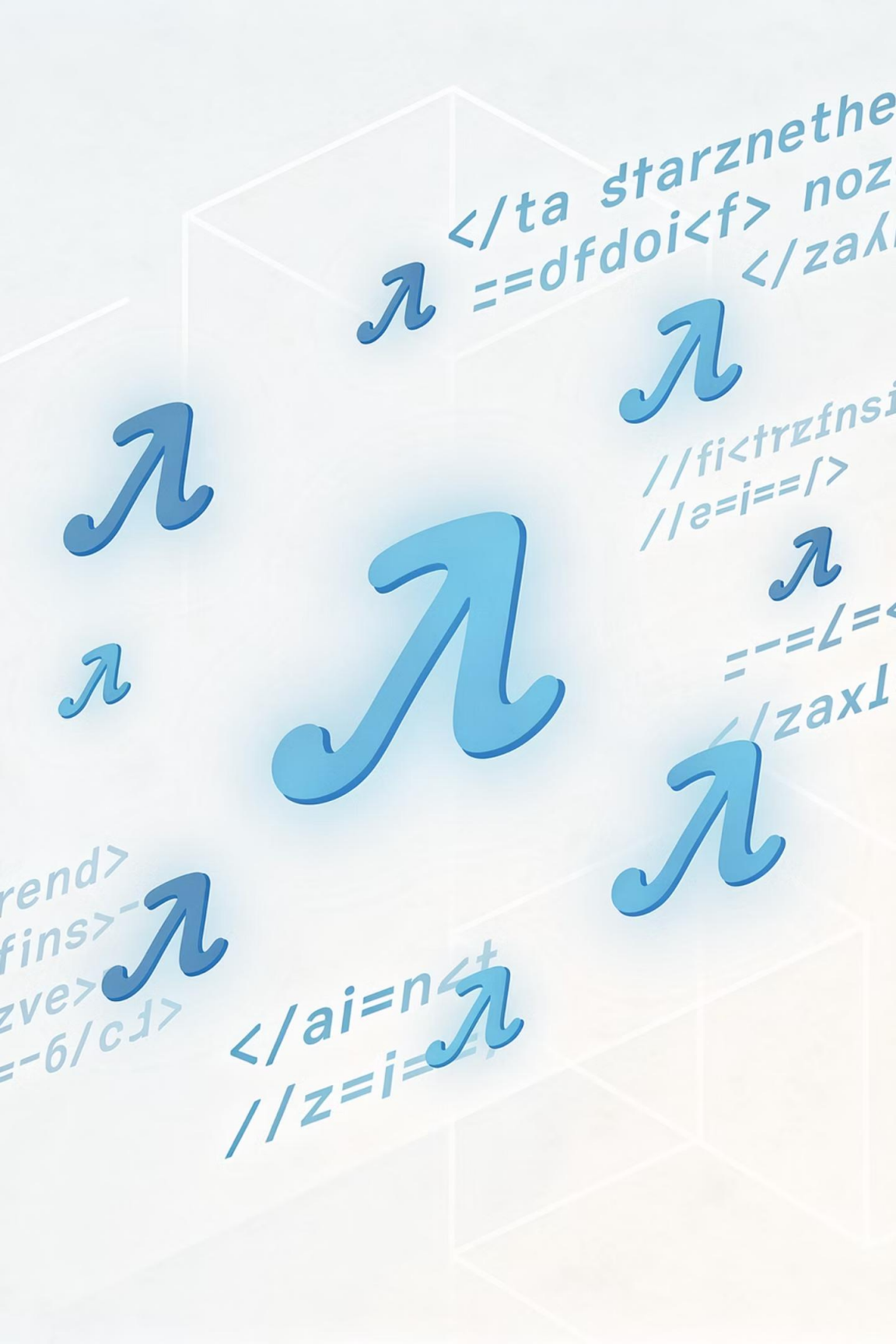


CS 331 Tutorial # 9

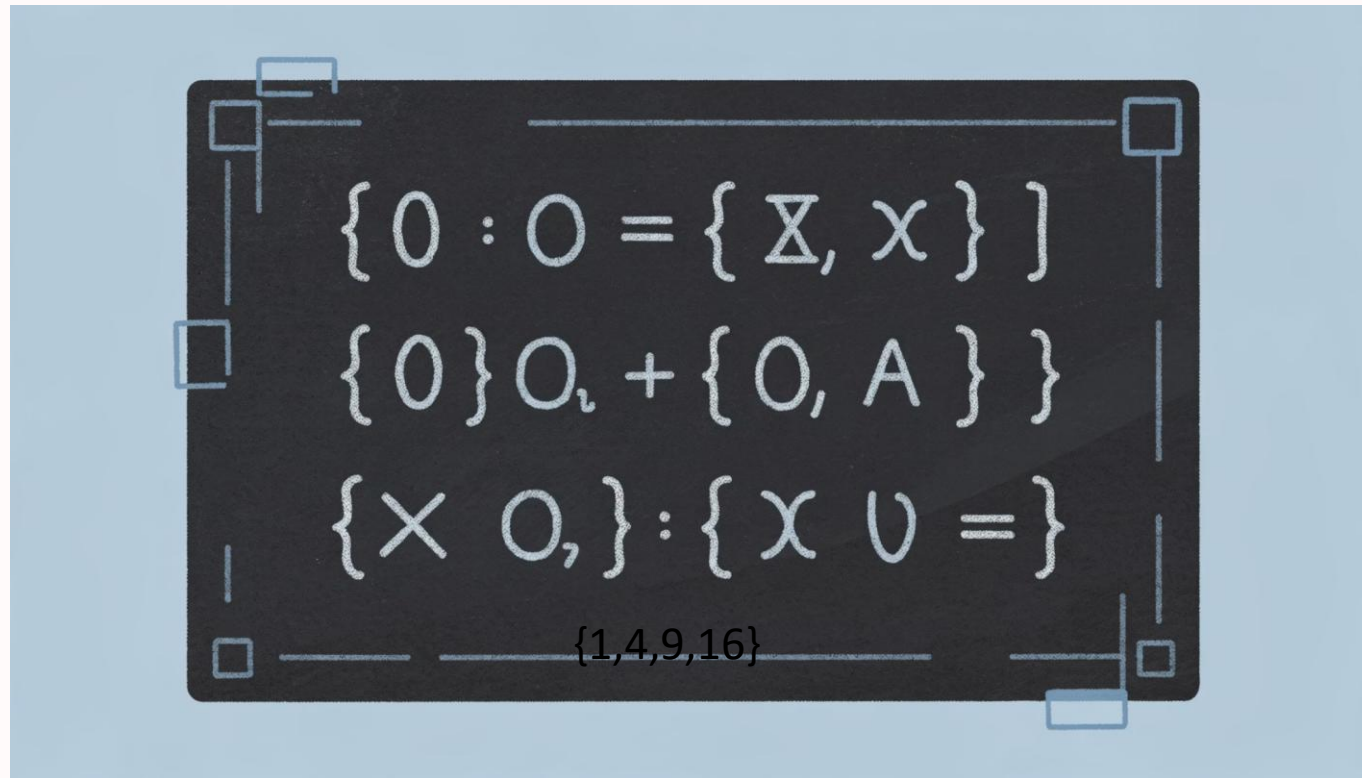
Functional Programming in Haskell





Haskell List Comprehensions

A deep dive into one of Haskell's most elegant features — inspired by mathematics, designed for clarity.



CONCEPT

What are List Comprehensions?

List comprehensions provide a **concise, expressive** way to create new lists from existing ones. Borrowed directly from mathematical **set-builder notation**, they let you define what a list should contain — including filtering and transformation — all in a single, readable expression.

`{ x^2 | x ∈ {1,2,3,4}, x<=10 }` Set Comprehension `{1,4,9}`

→ Generate

Build lists from existing ones

→ Filter

Apply conditions to include only matching elements

→ Transform

Apply an expression to produce new values

SYNTAX

The Basic Structure

```
[ expression | generator, condition1, condition2, ... ]
```

1

Expression

Defines **what each element** of the resulting list will look like. This is evaluated for every element that passes the conditions.

2

Generator

Draws elements from an existing list using the `<-` syntax — e.g. `x <- [1,2,3]` binds each element to `x` in turn.

3

Condition

Optional predicates that **filter elements**. Only elements for which all conditions evaluate to **True** are included.

EXAMPLE 1

Simple Example: Doubling Numbers

The simplest list comprehension applies a transformation to every element of an input list.

Goal

Double every element in
`[1,2,3,4]`

Code

```
[x*2 | x <- [1,2,3,4]]
```

Result

```
[2, 4, 6, 8]
```

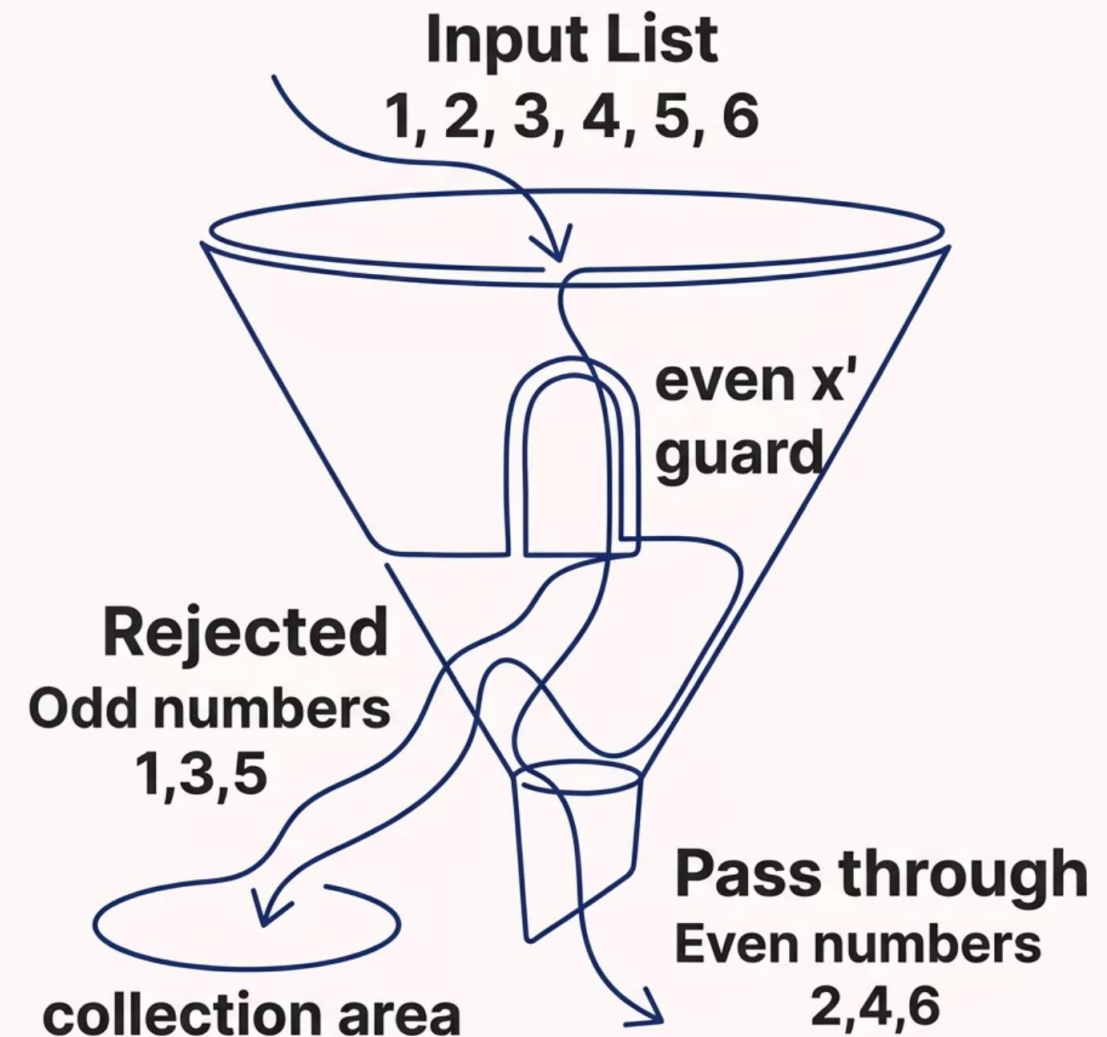
01230300120488
01228004120888
01230300130888
01238400130488
01230300120488
01230800130888
01230304120288

Adding Conditions: Filtering Even Numbers

Conditions (also called **guards**) act as filters inside a list comprehension. Only elements for which the predicate returns **True** are included in the output list.

```
[x | x <- [1,2,3,4,5,6], even x]  
-- Result: [2,4,6]
```

Here, **even x** is the guard. It tests each value of **x** drawn from the input list, silently discarding odd numbers and retaining only the even ones.



EXAMPLE 3

Multiple Generators: Cartesian Product

You can combine **two or more generators** in a single comprehension. Haskell iterates through each combination — the rightmost generator changes fastest, producing a full Cartesian product.

```
[(x,y) | x <- [1,2,3], y <- [4,5,6]]
```

Result: `[(1,4),(1,5),(1,6),(2,4),(2,5),(2,6),(3,4),(3,5),(3,6)]`

Outer Generator

`x <- [1,2,3]` — iterates once per value

Inner Generator

`y <- [4,5,6]` — iterates fully for *each* `x`

Output

9 pairs — every combination of both lists

EXAMPLE 4

Combining Generators and Conditions

The Code

```
[(x,y) | x <- [1,2,3],  
        y <- [4,5,6],  
        x < y]
```

Result

```
[(1,4),(1,5),(1,6),  
 (2,4),(2,5),(2,6),  
 (3,4),(3,5),(3,6)]
```

Here, the guard `x < y` is applied **after** both generators produce a candidate pair. Since every value drawn from `[4,5,6]` is already greater than every value in `[1,2,3]`, all pairs pass in this case.

This illustrates the key principle: generators define the **search space**, whilst conditions **prune** it — a clean separation of concerns.

List Comprehensions vs. **map** and **filter**

Transformation with **map**

`[x*2 | x <- [1,2,3,4]]` is equivalent to `map (*2) [1,2,3,4]`

Filtering with **filter**

`[x | x <- [1..6], even x]` is equivalent to `filter even [1,2,3,4,5,6]`

When to use which?

For simple transforms or filters, either works. List comprehensions shine when **combining** multiple generators and guards, as the syntax stays readable and mirrors mathematical intent.

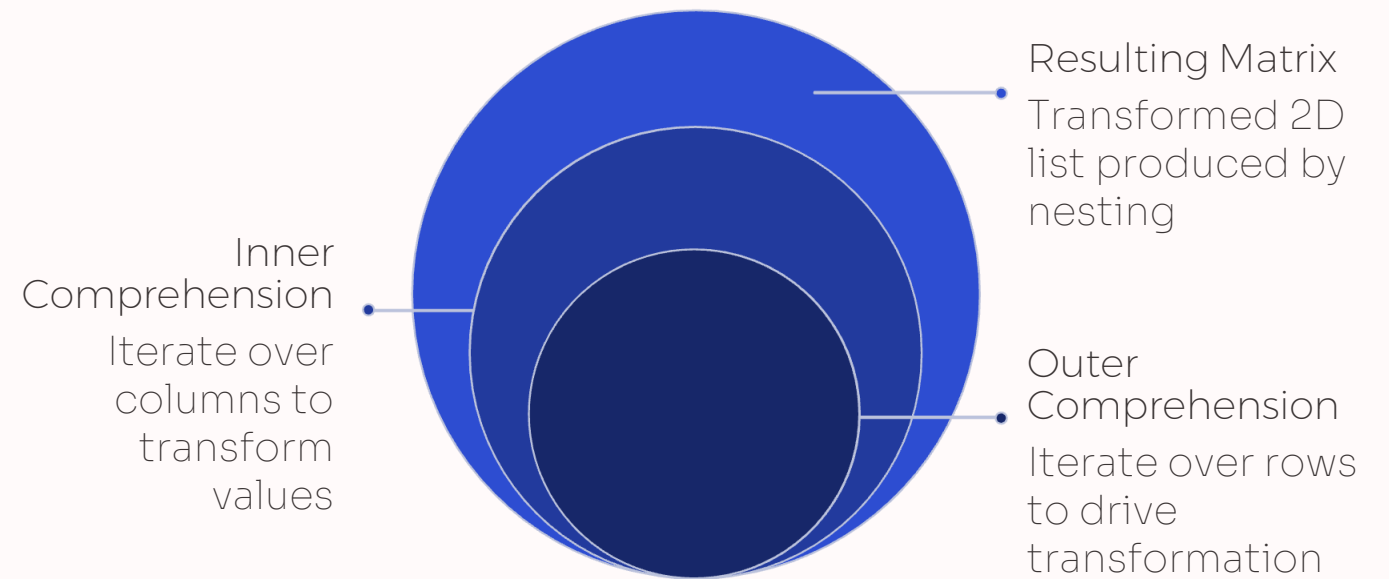
ADVANCED

Nested List Comprehensions

List comprehensions can be **nested** — placing one comprehension inside the expression or generator of another. This unlocks complex multi-dimensional transformations that would otherwise require multiple **map** and **concatMap** calls.

Common use cases include operating on **matrices** (lists of lists), generating structured data, or applying transformations to nested collections — all whilst retaining clean, readable syntax.

❏ Nesting should be used judiciously — deeply nested comprehensions can reduce readability. Extract named functions when logic becomes complex.



```
let xxs = [[1,3,5,2,3,1,2,4,5],  
          [1,2,3,4,5,6,7,8,9],  
          [1,2,4,2,1,6,3,1,3,2,3,6]]  
  
[ [ x | x <- xs, even x ] | xs <- xxs]
```

Result: [[2,2,4],[2,4,6,8],[2,4,2,6,2,6]]

Summary: The Power of List Comprehensions



Concise Syntax

Express complex list operations in a single, readable line



Filter & Transform

Seamlessly combine generators and guards in one expression



Mathematical Roots

Mirrors set-builder notation, making intent immediately clear



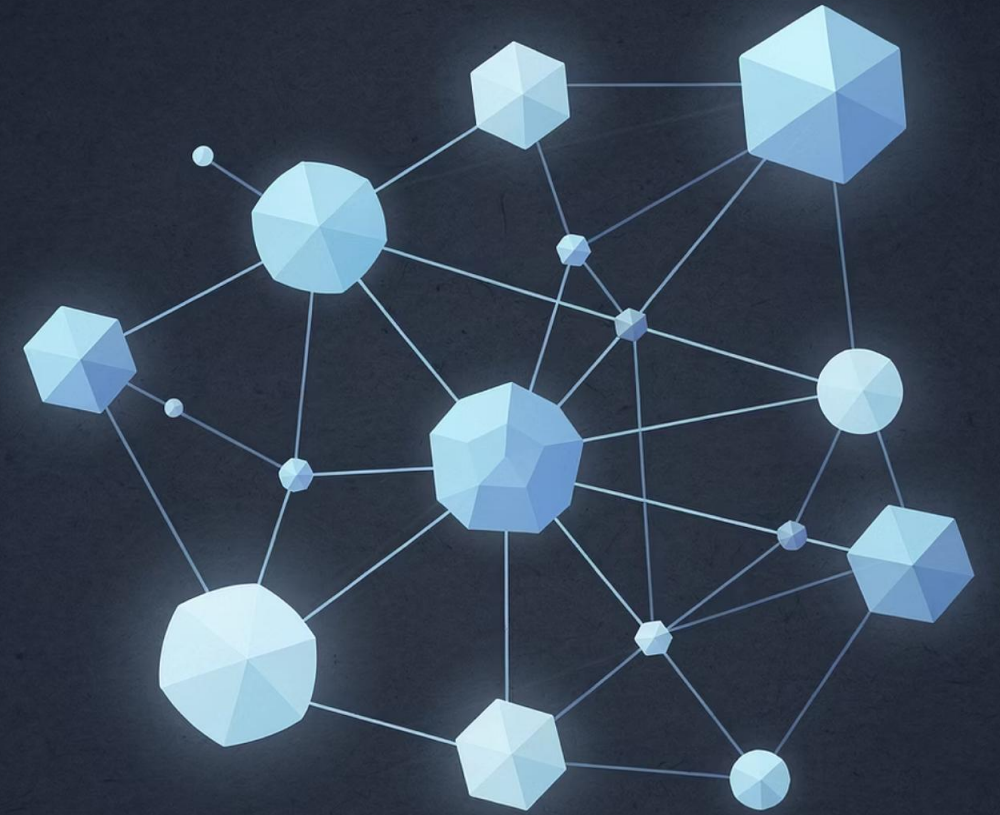
Functional Core

A fundamental building block of idiomatic Haskell programming



Haskell Type Classes

A deep dive into one of Haskell's most powerful features — enabling flexible, reusable, and type-safe code through the elegant mechanism of type classes.



The Problem: Overloading Operations

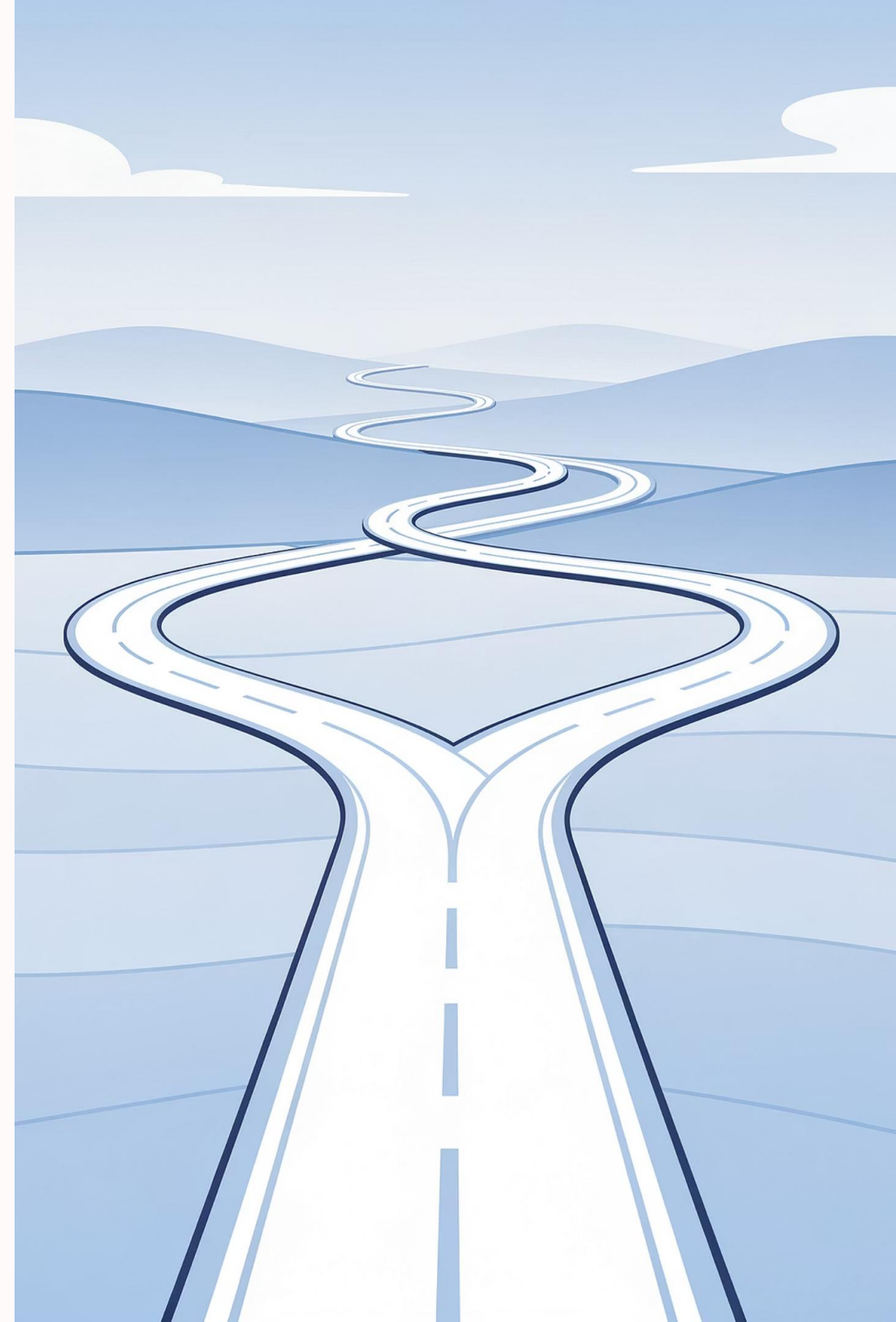
In most programming contexts, we want certain operations — like checking equality — to work across many different types. But the *way* equality is checked for integers is fundamentally different from how it works for lists, or custom data types.

Parametric Polymorphism

A single implementation works uniformly for all types. Example: `length :: [a] -> Int` counts elements regardless of their type — the behaviour never changes.

Ad-Hoc Polymorphism

Different types can have **different implementations** of the same function. The equality operator `==` is a perfect example — comparing two `Int` values is entirely different from comparing two lists. Haskell solves this with **type classes**.



Introducing Type Classes

Type classes are Haskell's elegant answer to ad-hoc polymorphism. **Ad-Hoc Polymorphism** allows the same function name (like == for equality or + for addition) to be used with values of different types,

Real world analogy

Concept	Example
Type class	“Drivable”
Types	Car, Bike
Function	drive
Instance	how each vehicle drives

A type class is a **collection of types that implement a common set of functions**, allowing polymorphism and overloading in Haskell.. Type class defines a set of operations that multiple types can implement, enabling polymorphism and function overloading in a type-safe way.

Defining a Type Class: The `Eq` Example

The `Eq` type class is one of the most fundamental in Haskell. It defines what it means for values of a type to be comparable for equality.

```
class Eq a where
  (==) :: a -> a -> Bool
  (/=) :: a -> a -> Bool
```

This declaration states: for any type `a` to be a member of `Eq`, it *must* provide implementations of both `==` and `/=`. In practice, defining one is often sufficient, as each has a sensible default in terms of the other.

Key Insight

The type variable `a` is a placeholder. The class definition says nothing about *how* equality is checked — only that it *must* be checkable. Each type provides its own logic.

Default Methods

Haskell allows default implementations inside a class. For `Eq`, you can define `/=` as `not (x == y)`, so instances only need to implement `==`.

Becoming an Instance: **Int** and **Bool**

Once a class is defined, individual types opt in by declaring an **instance**. Each instance provides the concrete implementation required by the class contract.

instance Eq Int

```
instance Eq Integer where
  x == y = x `integerEq` y
```

Compares two integer values using low-level machine equality. The implementation is type-specific and efficient.

instance Eq Bool

```
instance Eq Bool where
  True  == True  = True
  False == False = True
  _     == _     = False
```

Pattern matching on the two Boolean constructors gives a clear, exhaustive definition.

Using Constraints

```
elem :: Eq a =>
  a -> [a] -> Bool
```

The **Eq a =>** constraint means: "this function works for *any* type **a**, provided it is an instance of **Eq**."

The type Integer is an instance of the class Eq, and here is the definition of the method corresponding to the operation ==.

Everything before the ==> symbol is called a **class constraint**. We can read the type declaration like this: the elem function takes any two values where second is a list and both are of the same type and returns a **Bool**. The type of those two values must be a member of the **Eq** class (this was the class constraint).

The **Ord** Class: Ordering Values

The **Ord** class extends **Eq**, adding the ability to compare values by magnitude or ordering. This is expressed through a superclass constraint.

```
class Eq a => Ord a where
  compare :: a -> a -> Ordering
  (<), (<=), (>), (>=)
    :: a -> a -> Bool
```

The **Ordering** type has three constructors — **LT**, **EQ**, and **GT** — representing less than, equal to, and greater than. The comparison operators are typically derived from **compare**.

```
bigger :: Ord a => a -> a -> a
bigger x y = if x > y then x else y
```

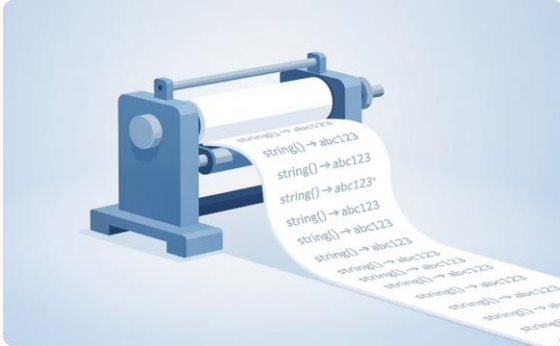
Superclass Constraint

Because **Ord** requires **Eq**, any type that is an instance of **Ord** is automatically also an instance of **Eq**. This creates a hierarchy of classes.

Built-in Instances

Types like **Int**, **Char**, **Double**, and **String** are all instances of **Ord**, enabling sorting, searching, and range operations.

Show and Read: Converting To and From Strings



The Show Class

Allows any value to be rendered as a human-readable **String**.

```
class Show a where  
  show :: a -> String
```

Example: `show 123` produces `"123"`. Custom types can derive or implement **Show** for debugging and logging.



The Read Class

The inverse of **Show** — parses a **String** and reconstructs a typed Haskell value.

```
class Read a where  
  read :: String -> a
```

Example: `read "123" :: Int` returns the integer `123`. A type annotation is often required to disambiguate the target type.

Enum and Bounded: Sequences and Limits

These two classes extend Haskell's expressive power for types that have a natural ordering with discrete steps, or finite ranges.

The Enum Class

Enum members are sequentially ordered types — they can be enumerated. They also have defined successors and predecessors, which you can get with the `succ` and `pred` functions. Types in this class: `()`, `Bool`, `Char`, `Ordering`, `Int`, `Integer`, `Float` and `Double`.

```
ghci> ['a'..'e']  
"abcde"
```

```
ghci> [3 .. 5]  
[3,4,5]
```

```
ghci> succ 'B'  
'C'
```

The Bounded Class

Declares that a type has a **minimum** and **maximum** value, accessible via `minBound` and `maxBound`.

Bounded Types

`Int`, `Char`, `Bool` — all have well-defined bounds.

`maxBound :: Int` gives the largest representable integer.

Unbounded Types

`Integer` (arbitrary precision) and `Float` are *not* instances of `Bounded`, as they have no fixed limits.

The Power of Type Classes

Type classes are not merely a syntactic convenience — they are a fundamental design philosophy that makes Haskell code robust, composable, and future-proof.



Code Reusability

Write a function once with a class constraint, and it works for *every* type that satisfies it. `elem :: Eq a => a -> [a] -> Bool` works on lists of integers, strings, or any custom type with an `Eq` instance.



Abstraction

Type classes define common **interfaces** without specifying implementation. This separates the *what* (the contract) from the *how* (the instance), promoting clean architectural boundaries.

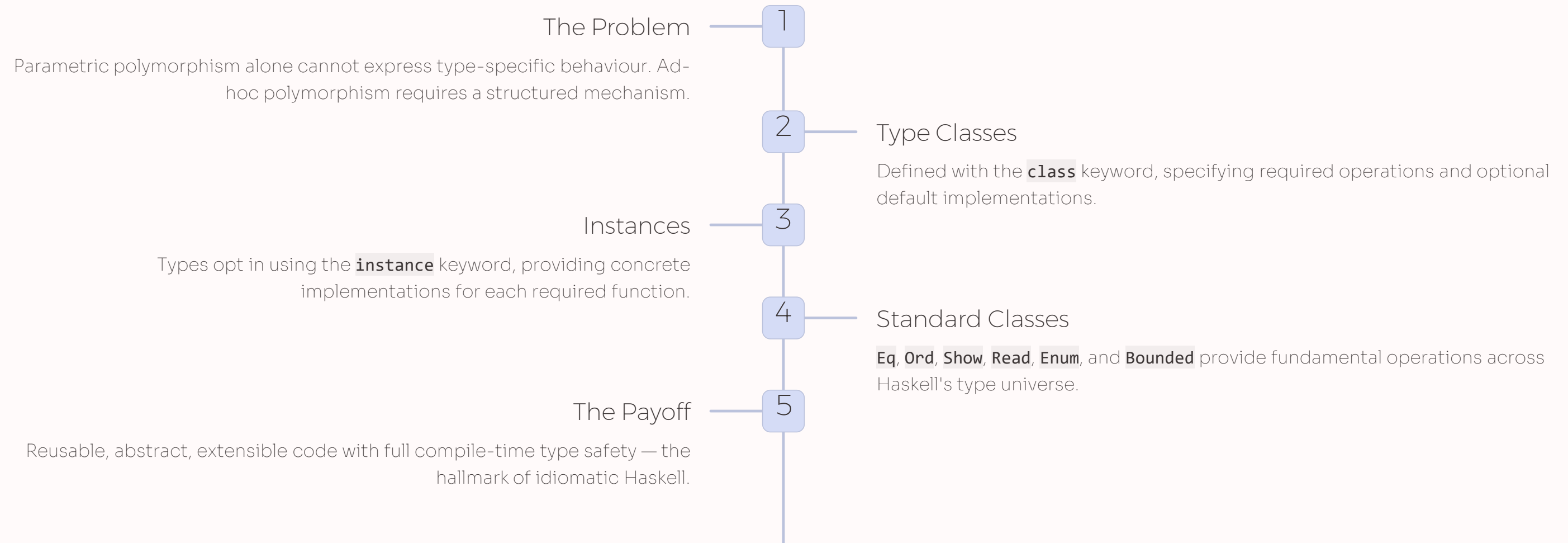


Extensibility

Any new type can be made an instance of an existing class, and new classes can be defined freely. The system is open and composable — existing code does not need to change.

Summary: Type Classes in Haskell

Type classes are the backbone of Haskell's standard library and its approach to polymorphism. Here is a consolidated view of what we have covered.



□ Type classes can also be **derived automatically** using the **deriving** keyword. For example, **deriving (Eq, Ord, Show)** after a data declaration tells the compiler to generate standard instances without manual implementation.

Haskell IO: Bridging the Pure and the Real World

A deep dive into how Haskell — a purely functional language — manages to interact with the real world without sacrificing its core principles of purity and referential transparency.



The Mind-Body Problem in Functional Programming

The Pure World

Pure functions are the cornerstone of Haskell. Given the same input, they **always** produce the same output. There are no hidden state changes, no global mutations, and absolutely no side effects. This predictability is what makes Haskell code so easy to reason about and test.

The Real World

Useful programs *must* interact with the outside world — reading from standard input, printing results, accessing the filesystem, making network calls. The challenge: how can a purely functional language do all of this without fundamentally breaking its own rules?

- This tension — between mathematical purity and practical necessity — is one of the most fascinating design problems in programming language theory.

$$E - (x^2 + x - C+)$$

$$r + y^2 + = Cy^2$$

$$I + \frac{b}{Z} = x^2 Y \frac{Kx}{x' - 2}$$

$$a$$

$$S + < l^{21} + \Sigma - O$$

$$x \quad N = \frac{F}{Z} = O$$

$$+ \quad \quad \quad + - l^2$$

$$\mu^2 - B^2$$

$$x = Z - 1/6$$

The Monadic Solution: Introducing the **IO** Type

The key insight of Haskell's monadic I/O is the distinction between **describing** an action and **performing** it. The **IO** type encapsulates this idea elegantly.

What is **IO a**?

An **IO a** value is a **plan** or **recipe** for an action. When performed by the runtime, it will produce a value of type **a** and may cause side effects. Crucially, constructing an **IO a** value does *not* immediately cause any effects.

A Concrete Example

putChar :: Char -> IO () takes a character and returns an **IO ()** action. Calling **putChar 'x'** builds a description of the action of printing **'x'**. Nothing is printed until that description is executed by the runtime system.

Preserving Purity

Because **IO** actions are just values, they can be passed around, stored in data structures, and combined — all without breaking referential transparency. The type system enforces a clean boundary between pure and effectful code.

Performing Actions: `main` and GHCi

The `main` Entry Point

Every Haskell executable has a designated entry point: `main :: IO ()`. When you run a compiled Haskell program, the runtime system **executes** the `main` action. All I/O that your program performs must ultimately be reachable from `main`. This single entry point is where the "pure world" hands off control to the "real world".

GHCi: Interactive Execution

In GHCi, the interactive REPL, you can type an `IO` action directly and GHCi will execute it on your behalf — acting as a surrogate `main`. This makes it easy to test I/O actions in isolation during development.

The Critical Distinction

Evaluating an `IO` action — computing what the action *is* — never causes side effects.

Executing an `IO` action — handing it to the runtime — is what actually performs the effect.

This separation is what makes Haskell's approach to I/O both principled and safe.

do Notation: Syntactic Sugar for Monads

Writing long chains of `>>` and `>>=` quickly becomes unwieldy. Haskell's `do` notation provides a clean, imperative-looking syntax that desugars to these monadic operators automatically.

1

Binding Results: `x <- action`

Executes `action` and binds its resulting value to the name `x`. This desugars to the `>>=` (bind) operator. Use this whenever you need to use the result of an I/O action in subsequent code.

2

Discarding Results: `action`

Executes `action` purely for its side effect, discarding its result. This desugars to the `>>` operator. Use this for actions like `putStrLn` where the return value `()` is not needed.

3

Returning Pure Values: `return`

The `return` function wraps a pure value into the `IO` monad without performing any side effects. It is the final piece needed to complete the `Monad` typeclass interface alongside `>>=`.

main.hs

```
main :: IO ()
main = do
    putStrLn "Enter your name:"
    name <- getLine
    putStrLn ("Hello, " ++ name ++ "!")

    putStrLn "Enter a number:"
    numStr <- getLine
    let num = read numStr :: Int
    print (num * 2)
```

Using IO directly in GHCi

```
ghci> name <- getLine
Richa
ghci> putStrLn ("Hi " ++ name)
Hi Richa
```


Key **IO** Actions and Their Types

Haskell's standard library provides a rich set of primitive **IO** actions. Understanding their types is essential to writing correct I/O code.



getChar

```
getChar :: IO Char
```

Reads a single character from standard input and returns it as an **IO Char** action.



putStrLn

```
putStrLn :: String -> IO ()
```

Prints a **String** to standard output, appending a newline character at the end.



readFile

```
readFile :: FilePath -> IO String
```

Lazily reads the entire content of a file at the given path, returning it as a **String**.



writeFile

```
writeFile :: FilePath -> String -> IO ()
```

Writes a **String** to the specified file, creating it if it does not exist or overwriting it if it does.

Why **IO** is Different

Not Just a Data Structure

Familiar Haskell types like `[]` are algebraic data structures — they hold values and can be pattern-matched. **IO** is fundamentally different: it represents a **computation that interacts with the external world**. You cannot inspect the "inside" of an **IO** action from pure code, by design.

Deferred Execution

The runtime system is the only entity that actually *runs* **IO** actions. Everything you write in Haskell merely **describes** what should happen. Execution is deferred until the runtime processes the **main** action, maintaining referential transparency throughout the rest of the program.

The Key Guarantee

Pure functions remain **purely functional** — they cannot accidentally perform I/O.

Effectful code is **clearly marked** by the **IO** type.

The type system **enforces** this boundary at compile time — no runtime checks needed.

The Power of Explicit Effects

Haskell's approach to I/O is often cited as one of its most elegant and influential contributions to programming language design — solving what Simon Peyton Jones called the *"awkward squad"* of real-world programming.



Equational Reasoning

Because pure functions have no hidden effects, you can substitute any expression with its value freely — just like in mathematics. This makes refactoring, testing, and formal verification dramatically more tractable than in imperative languages.



Explicit Boundaries

The type system clearly demarcates where interactions with the real world occur. Any function returning **IO a** is, by definition, effectful. Any function not involving **IO** is guaranteed to be pure. There is no ambiguity and no surprises.



Composable and Principled

Side effects in Haskell are not a necessary evil — they are first-class, composable values. **IO** actions can be combined, abstracted over, stored in data structures, and reasoned about with the same rigour as any other Haskell value.



📌 Haskell's **IO** monad has directly influenced modern languages including Rust, Scala, and Swift in their approaches to managing side effects safely.